3rd,BCNF, 4th, 5th NF

- **Types of Functional Dependencies**

1. Trivial functional dependency

- The dependency of an attribute on a set of attributes is known as trivial functional dependency if the set of attributes includes that attribute.
- A ->B is trivial functional dependency if B is a subset of A.
- Consider a table with two columns Student_id and Student_Name.
- {Student_Id, Student_Name} -> Student_Id is a trivial functional dependency as Student_Id is a subset of {Student_Id, Student_Name}.

2. Non trivial functional dependency

- If a functional dependency X->Y holds true where Y is not a subset of X then this dependency is called non trivial Functional dependency.
- Example:
- An employee table with three attributes: emp_id, emp_name, emp_address:
- emp_id -> emp_name (emp_name is not a subset of emp_id)
  emp_id -> emp_address (emp_address is not a subset of emp_id)
- {emp_id, emp_name} -> emp_name => What is this FD?
- **Completely non trivial FD**:
- If a FD X->Y holds true where X intersection Y is null then this dependency is said to be completely non trivial function dependency.

3. Transitive dependency

- A functional dependency is said to be transitive if it is indirectly formed by two functional dependencies.
- X -> Z is a transitive dependency if the following three functional dependencies hold true:
  - X->Y
  - Y does not ->X
  - Y->Z
- **Note:** A transitive dependency can only occur in a relation of three of more attributes.
- **Example**:

Book	Author	Author_age
Game of Thrones	George R. R. Martin	66
Harry Potter	J. K. Rowling	49
Dying of the Light	George R. R. Martin	66

- {Book} ->{Author} (if we know the book, we knows the author name)
- {Author} does not ->{Book}
- {Author} -> {Author_age}

- Having introduced functional dependencies, we are now ready to use them to specify some aspects of the semantics of relation schemas

4. Multivalued dependency

- Occurs when there are more than one independent multivalued attributes in a table.
- Consider a bike manufacture company, which produces two colors (Black and Red) in each model every year.

bike_model	manuf_year	color
M1001	2007	Black
M1001	2007	Red
M2012	2008	Black
M2012	2008	Red
M2222	2009	Black
M2222	2009	Red

Here columns manuf_year and color are independent of each other and dependent on bike_model.
In this case these two columns are said to be multivalued dependent on bike_model.
bike_model ->> manuf_year
bike_model ->> color

Second Normal Form

- A table is in 2NF iff
  - It is in 1NF and
  - no non-prime attribute is dependent on any proper subset of any candidate key of the table
- A ***non-prime attribute*** of a table is an attribute that is not a part of any candidate key of the table
- A ***candidate key*** is a minimal superkey

Steps to Normalize to 2NF

1.Ensure the table is in 1NF

1. All attributes contain atomic values (no repeating groups or arrays).

2.Identify partial dependencies

1. Check for composite primary keys.
2. Find attributes that depend on only part of the key.

3.Remove partial dependencies

1. Create new tables for the partially dependent attributes.
2. Keep only attributes fully dependent on the entire primary key in the original table.
3. Establish **foreign keys** to maintain relationships.

1. STUDENT Table

StudentID	StudentName
S1	Ravi
S2	Anita

2. COURSE Table

CourseID	CourseName
C1	DBMS
C2	Networks

3. ENROLLMENT Table

StudentID	CourseID	Grade
S1	C1	A
S2	C1	B
S1	C2	A+

Third Normal Form

- A relation is in third normal form if it holds atleast one of the following conditions for every non-trivial function dependency X → Y.
- X is a super key.
- Y is a prime attribute, i.e., each element of Y is part of some candidate key.

In other words

- *A relation that is in First and Second Normal Form and in which no non-primary-key attribute is transitively dependent on the primary key, then it is in Third Normal Form (3NF).*

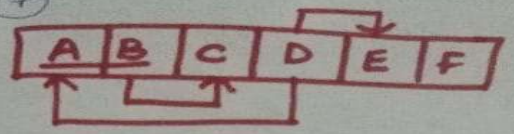
Third Normal Form

- A relation will be in 3NF if it is in 2NF and not contain any transitive dependency.
- 3NF is used to reduce the data duplication. It is also used to achieve the data integrity.
- If there is no transitive dependency for non-prime attributes, then the relation must be in third normal form.

- **Steps to Normalize to 3NF**
- **Ensure the table is in 2NF** (no partial dependencies).
- **Identify transitive dependencies**:
  - Find attributes that depend on other non-key attributes.
- **Remove transitive dependencies**:
  - Move those attributes into a separate table.
  - Connect tables using **foreign keys**.

- If it is not in 3NF, decompose the Table R
- For ex: if X-> Y is TD
- Then decompose R as
- R1 with(X^+)
- R2 with (R-Y^+)

Example (7)

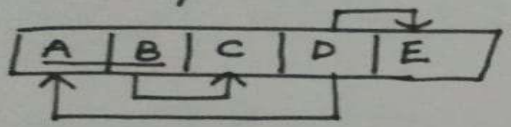
given R 

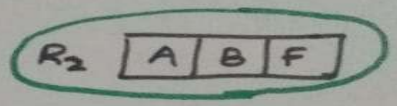
F is a multivalued attribute. Is R in 3NF?

Solution:

R is not in 1NF as F is not atomic.

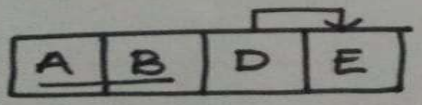
So decompose R to R_1 and R_2 where

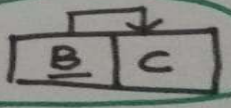
R_1 

and R_2 

Now R_2 is in 1NF and 2NF

R_1 is in 1NF but not in 2NF because of PD $B \rightarrow C$. So decompose R_1 to R_{11} and R_{12}

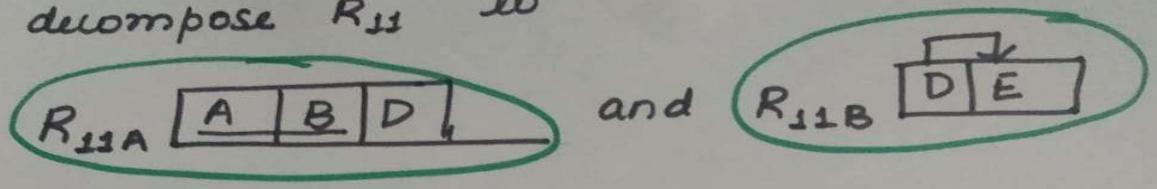
R_{11} 

R_{12} 

R_{12} satisfies 1NF, 2N & 3NF

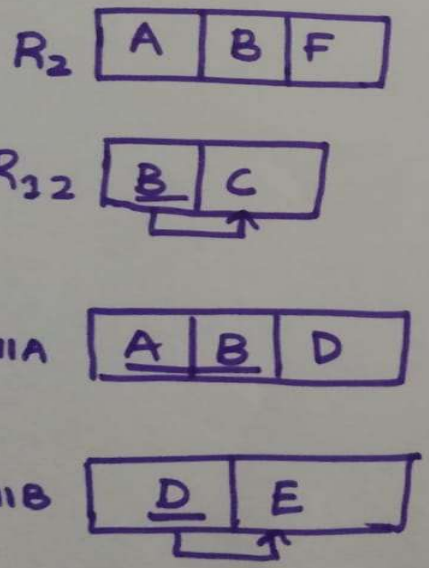
R_{11} satisfies 1NF and 2NF but not 3NF because of TD. $AB \rightarrow D$ $D \rightarrow E$ then $AB \rightarrow E$

So decompose R_{11} To



Now R_{11A} and R_{11B} satisfy 1NF, 2NF and 3NF.

Therefore the decomposed tables are



- **Example 1: Employee Table (Not in 3NF)**
- **EMPLOYEE Table : Primary Key**: EmpID, Depid-> Depname, Deploc

EmpID	EmpName	DeptID	DeptName	DeptLocation
E1	Ravi	D1	HR	Mumbai
E2	Anita	D2	IT	Bangalore
E3	John	D1	HR	Mumbai

- **Analysis**
- The table is in **2NF** (single-attribute key, so no partial dependencies).
- But **DeptName** and **DeptLocation** depend on **DeptID**, not directly on EmpID → **transitive dependency**.

- **Step 1: Decompose**
- **1. EMPLOYEE Table (Keep attributes depending on EmpID)**

EmpID	EmpName	DeptID
E1	Ravi	D1
E2	Anita	D2
E3	John	D1

- **2. DEPARTMENT Table** (Move Dept-related details

DeptID	DeptName	DeptLocation
D1	HR	Mumbai
D2	IT	Bangalore

- **Exercise**
- 1. Given the following SALES table, identify transitive dependencies and normalize it to 3NF:

SaleID	ProductID	ProductName	Category	SaleDate
S01	P01	Laptop	Electronics	2025-09-10
S02	P02	Chair	Furniture	2025-09-11
S03	P01	Laptop	Electronics	2025-09-12

Key Takeaways

- **3NF eliminates transitive dependencies.**
- Every non-key attribute should depend **only on the key**.
- Use **foreign keys** to preserve relationships between decomposed tables.
- After 3NF, you can consider **BCNF** (Boyce-Codd Normal Form) if overlapping candidate keys cause anomalies.

BCNF(Boyce Codd Normal Form) / 3.5 NF

BCNF **(Boyce Codd Normal Form)** is an advanced version of the third normal form (3NF), and often, it is also known as the 3.5 Normal Form.

Any one of the two conditions should be satisfied for a Table/ Relation satisfy BCNF.

i. X→Y should be a trivial dependency.
ii. X should be a super key.

X -> Y is a trivial functional dependency if Y is a subset of X

A table or relation is said to be in BCNF in DBMS if the table or the relation is already in 3NF, and also, for every functional dependency (let's say, X->Y), X is either the super key or the candidate key. In simple terms, for any case (let's say, X->Y), X can't be a non-prime attribute.

In this example, we have a relation R with three columns: Id, Subject, and Professor. We have to find the highest normalization form, and also, if it is not in BCNF, we have to decompose it to satisfy the conditions of BCNF.

Id	Subject	Professor
101	Java	Mayank
101	C++	Kartik
102	Java	Sarthak
103	C#	Lakshay
104	Java	Mayank

Interpreting the table:

- One student can enroll in more than one subject.
  - Example: student with **Id 101** has enrolled in **Java and C++**.
- Professor is assigned to the student for a specified subject, and there is always a possibility that there can be multiple professors teaching a particular subject.

Finding the solution:

- Using Id and Subject together, we can find all unique records and also the other columns of the table. Hence, the Id and Subject together form the primary key.
- The table is in 1NF because all the values inside a column are atomic and of the same domain.
- We can't uniquely identify a record solely with the help of either the Id or the Subject name. As there is no partial dependency, the table is also in 2NF.
- There is no transitive dependency because the non-prime attribute i.e., Professor, is not deriving any other non-prime attribute column in the table. Hence, the table is also in 3NF.
- There is a point to be noted that the table is not in **BCNF (Boyce-Codd Normal Form)**.

Why is the table not in BCNF?

- As we know that each professor teaches only one subject, but one subject may be taught by multiple professors. This shows that there is a dependency between the subject & the professor, and the subject is always dependent on the professor **(professor -> subject).** As we know that the professor column is a non-prime attribute, while the subject is a prime attribute. This is not allowed in BCNF in DBMS. **For BCNF, the deriving attribute (professor here) must be a prime attribute.**

How to satisfy BCNF?

- R(SId, Sub, Prof)

- R1(Prof, Sub)
- R2(SId, Prof)

- Professor is now the primary key and the prime attribute column, deriving the subject column. **Hence, it is in BCNF.**

Consider the following relationship : **R (A,B,C,D)**

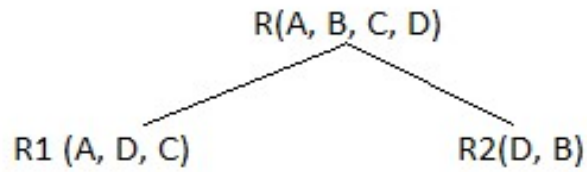
and following dependencies :

A -> BCD
BC -> AD
D -> B

Above relationship is already in 3rd NF. Keys are **A** and **BC.**

Hence, in the functional dependency, **A -> BCD**, A is the super key.
in second relation, **BC -> AD**, BC is also a key.
but in, **D -> B**, D is not a key.

Hence we can break our relationship R into two relationships **R1** and **R2.**



Breaking, table into two tables, one with A, D and C while the other with D and B.

- Let's take another general example to understand the concept of decomposition in detail: We have a relation **R(A, B, C, D)** that is already in 3NF.
- Candidate Keys: **{A, BC}**
- Prime Attributes: **{A, B, C}** Non-Prime Attributes: **{D}**
- Functional dependencies are as follows: {A->BCD, BC->AD, D->B}
- The above relation is not in BCNF because {D->B} is not in BCNF as {D} is neither a candidate key nor a prime attribute. Hence, we will decompose the relation R into R1{A, D, C} and R2{D, B}.

- BCNF decomposition does not always satisfy dependency preserving property.
- After BCNF decomposition if dependency is not preserved then we have to decide whether we want to remain in BCNF or rollback to 3NF.
- This process of rollback is called denormalization.

Example

- Library allows patrons to request books that are currently out

BookNo	Patron	PhoneNo
B3	J. Fisher	555-1234
B2	J. Fisher	555-1234
B2	M. Amer	555-4321

Example

- Candidate key is {BookNo, Patron}
- We have
  - Patron → PhoneNo
- Table is not 2NF
  - Potential for
    - Insertion anomalies
    - Update anomalies
    - Deletion anomalies

2NF Solution

- Put telephone number in separate Patron table

BookNo	Patron
B3	J. Fisher
B2	J. Fisher
B2	M. Amer

Patron	PhoneNo
J. Fisher	555-1234
M. Amer	555-4321

Third Normal Form

- A table is in 3NF iff
  - it is in 2NF and
  - all its attributes are determined only by its candidate keys and not by any non-prime attributes

Example

- Table BorrowedBooks

BookNo	Patron	Address	Due
B1	J. Fisher	101 Main Street	3/2/15
B2	L. Perez	202 Market Street	2/28/15

- Candidate key is BookNo, Patron
- Patron $\rightarrow$ Address

3NF Solution

- Put address in separate Patron table

BookNo	Patron	Due
B1	J. Fisher	3/2/15
B2	L. Perez	2/28/15

Patron	Address
J. Fisher	101 Main Street
L. Perez	202 Market Street

Boyce-Codd Normal Form

- Stricter form of 3NF
- A table *T* is in BCNF iff
  - for every one of its non-trivial dependencies $X \rightarrow Y$, *X* is a super key for *T*

- Most tables that are in 3NF also are in BCNF

Example: Let's assume there is a company where employees work in more than one department.

EMPLOYEE table:

EMP_ID	EMP_COUNTRY	EMP_DEPT	DEPT_TYPE	EMP_DEPT_NO
264	India	Designing	D394	283
264	India	Testing	D394	300
364	UK	Stores	D283	232
364	UK	Developing	D283	549

In the above table Functional dependencies are as follows:

> EMP_ID → EMP_COUNTRY
> EMP_DEPT → {DEPT_TYPE, EMP_DEPT_NO}

Candidate key: {EMP-ID, EMP-DEPT}

The table is not in BCNF because neither EMP_DEPT nor EMP_ID alone are keys.
To convert the given table into BCNF, we decompose it into three tables:

EMP_COUNTRY table:

EMP_ID	EMP_COUNTRY
264	India
264	India

EMP_DEPT table:

EMP_DEPT	DEPT_TYPE	EMP_DEPT_NO
Designing	D394	283
Testing	D394	300
Stores	D283	232
Developing	D283	549

EMP_DEPT_MAPPING table:

EMP_ID	EMP_DEPT
D394	283
D394	300
D283	232
D283	549

The table is not in BCNF because neither EMP_DEPT nor EMP_ID alone are keys.
To convert the given table into BCNF, we decompose it into three tables:

Functional dependencies:

> EMP_ID → EMP_COUNTRY
> EMP_DEPT → {DEPT_TYPE, EMP_DEPT_NO}

Candidate keys:

For the first table: EMP_ID

For the second table: EMP_DEPT

For the third table: {EMP_ID, EMP_DEPT}

Now, this is in BCNF because left side part of both the functional dependencies is a key.

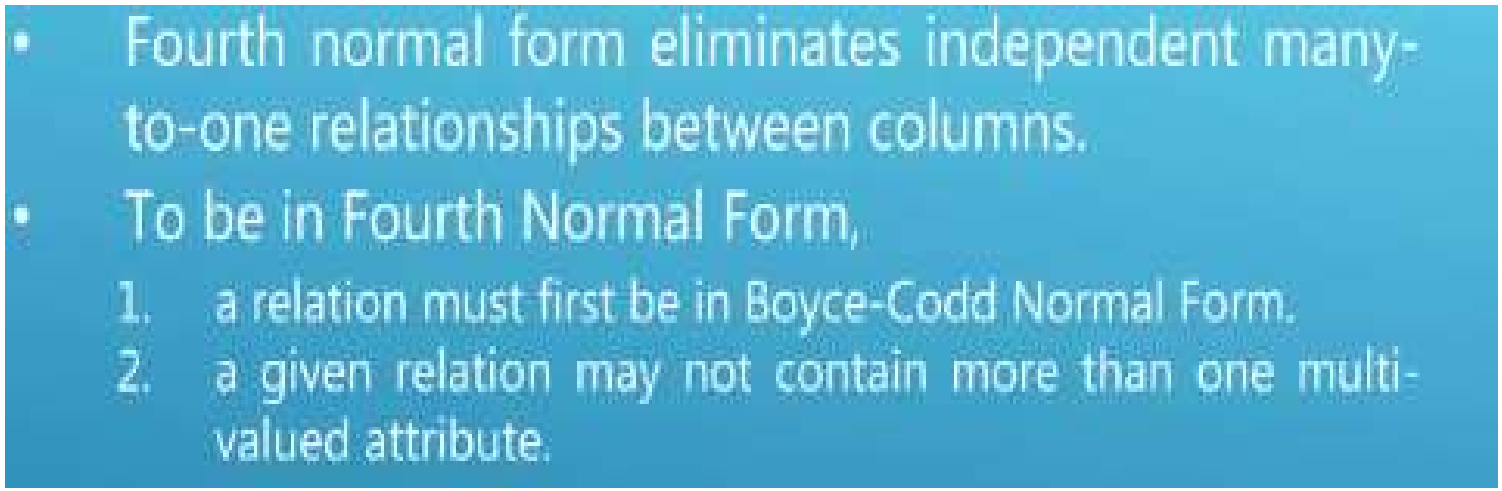
Fourth Normal Form

- Fourth Normal Form comes into picture when **Multi-valued Dependency** occur in any relation.
- we will learn about Multi-valued Dependency, how to remove it and how to make any table satisfy the fourth normal form.

Rules for 4th Normal Form

- For a table to satisfy the Fourth Normal Form, it should satisfy the following two conditions:
- It should be in the **Boyce-Codd Normal Form**.
- And, the table should not have any **Multi-valued Dependency**.

(For a dependency A → B, if for a single value of A, multiple values of B exists, then the relation will be a multi-valued dependency.)



What is Multi-valued Dependency?

A table is said to have multi-valued dependency, if the following conditions are true,

•For a dependency A → B, if for a single value of A, multiple value of B exists, then the table may have multi-valued dependency.

•Also, a table should have at-least 3 columns for it to have a multi-valued dependency.

•And, for a relation R(A,B,C), if there is a multi-valued dependency between, A and B, then B and C should be independent of each other.

If all these conditions are true for any relation(table), it is said to have multi-valued dependency.

Example

STUDENT

STU_ID	COURSE	HOBBY
21	Computer	Dancing
21	Math	Singing
34	Chemistry	Dancing
74	Biology	Cricket
59	Physics	Hockey

The given STUDENT table is in 3NF, but the COURSE and HOBBY are two independent entity. Hence, there is no relationship between COURSE and HOBBY.

- In the STUDENT relation, a student with STU_ID, **21** contains two courses, **Computer** and **Math** and two hobbies, **Dancing** and **Singing**. So there is a Multi-valued dependency on STU_ID, which leads to unnecessary repetition of data.
- So to make the above table into 4NF, we can decompose it into two tables:

STUDENT_COURSE

STU_ID	COURSE
21	Computer
21	Math
34	Chemistry
74	Biology
59	Physics

STUDENT_HOBBY

STU_ID	HOBBY
21	Dancing
21	Singing
34	Dancing
74	Cricket
59	Hockey

Example

CTX

Course	Teacher	Text
Physics	Prof. Arul	Basic Mechanics
Physics	Prof. Arul	Principles of options
Physics	Prof. Elakkia	Basic Mechanics
Physic	Prof. Elakkia	Principles of options
Maths	Prof. Arul	Basic Mechanics
Maths	Prof. Arul	Vector Analysis
Maths	Prof. Arul	Trigonometry

CT

Course	Teacher
Physics	Prof. Arul
Physics	Prof. Elakkia
Maths	Prof. Arul

cx

Course	Text
Physics	Basic Mechanics
Physics	Principles of options
Maths	Basic Mechanics
Maths	Vector Analysis
Maths	Trigonometry

Example (Not in 4NF)

Scheme → {MovieName, ScreeningCity, Genre)

Primary Key: {MovieName, ScreeningCity, Genre)

1. All columns are a part of the only candidate key, hence BCNF
2. Many Movies can have the same Genre
3. Many Cities can have the same movie
4. Violates 4NF

Movie	ScreeningCity	Genre
Hard Code	Los Angles	Comedy
Hard Code	New York	Comedy
Bill Durham	Santa Cruz	Drama
Bill Durham	Durham	Drama
The Code Warrier	New York	Horror

Movie	Genre
Hard Code	Comedy
Bill Durham	Drama
The Code Warrier	Horror

Movie	ScreeningCity
Hard Code	Los Angles
Hard Code	New York
Bill Durham	Santa Cruz
Bill Durham	Durham
The Code Warrier	New York

Scheme → {Employee, Skill, ForeignLanguage}

1. Primary Key → {Employee, Skill, Language }
2. Each employee can speak multiple languages
3. Each employee can have multiple skills
4. Thus violates 4NF

Employee	Skill	Language
1234	Cooking	French
1234	Cooking	German
1453	Carpentry	Spanish
1453	Cooking	Spanish
2345	Cooking	Spanish

Employee	Skill
1234	Cooking
1453	Carpentry
1453	Cooking
2345	Cooking

Employee	Language
1234	French
1234	German
1453	Spanish
2345	Spanish

Fifth Normal Form (5NF)

- A relation is in 5NF if it is in 4NF and not contains any join dependency and joining should be lossless.
- (A relation is said to have join dependency if it can be recreated by joining multiple sub relations and each of these sub relations has a subset of the attributes of the original relation.)
- 5NF is satisfied when all the tables are broken into as many tables as possible in order to avoid redundancy.
- 5NF is also known as Project-join normal form (PJ/NF).

Fifth Normal Form (5NF)

- A relation is in 5NF if it is in 4NF and not contains any join dependency and joining should be lossless.
- 5NF is satisfied when all the tables are broken into as many tables as possible in order to avoid redundancy.
- 5NF is also known as Project-join normal form (PJ/NF).

- Consider the relation **R** below having the schema R(supplier, product, consumer). The primary key is a combination of all three attributes of the relation.

supplier	product	consumer
S1	P1	C1
S1	P2	C1
S2	P1	C1
S3	P3	C3

supplier	product
S1	P1
S1	P2
S2	P1
S3	P3

consumer	product
C1	P1
C1	P2
C3	P3

supplier	consumer
S1	C1
S2	C1
S3	C3

Explanation:

- Table 2, Table 3 and Table 4 when joined yield the original table (Table 1). Hence join dependency exists in Table 1, therefore Table 1 is not in 5NF or PJNF. However Table 2, Table 3 and Table 4 satisfy 5NF as it has no join dependency and cannot be decomposed further (join dependency does not exists). But this might not be true in all cases i.e., when we combine the decomposed tables, the resultant table may not be equivalent to the original table, in that case the original table is said to be in 5NF provided it is already in 4NF. However, 5NF is not applied in practical scenarios and remains limited to theoretical concepts.

- **Definition of 2NF:** No non-prime attribute should be partially dependent on Candidate Key. i.e. there should not be a partial dependency from X → Y.
- **Definition of 3NF:** First, it should be in 2NF and if there exists a non-trivial dependency between two sets of attributes X and Y such that X → Y (i.e. Y is not a subset of X) then

1. Either X is Super Key
2. Or Y is a prime attribute.

- **Definition of BCNF:** First, it should be in 3NF and if there exists a non-trivial dependency between two sets of attributes X and Y such that X → Y (i.e., Y is not a subset of X) then

1. X is Super Key

- NOTE: If a table is in BCNF then it is in 3NF, 2NF, and 1NF, similarly if the table is in 3NF Then it is in 2NF and 1NF. Hence, we can say that if a table is in the higher normal form then by default it is in lower normal form.

Example 1

Find the highest normal form in R (A, B, C, D, E) under following functional dependencies.
ABC --> D CD --> AE
Important Points for solving above type of question.
1) It is always a good idea to start checking from BCNF, then 3 NF, and so on.

2) If any functional dependency satisfied a normal form then there is no need to check for lower normal form.
Candidate keys in the given relation are {ABC, BCD}
For example, ABC –> D is in BCNF (Note that ABC is a superkey), so no need to check this dependency for lower normal forms.

BCNF: ABC -> D is in BCNF. Let us check CD -> AE, CD is not a super key so this dependency is not in BCNF. So, R is not in BCNF.
3NF: ABC -> D we don't need to check for this dependency as it already satisfied BCNF. Let us consider CD -> AE. Since E is not a prime attribute, so the relation is not in 3NF.
2NF: In 2NF, we need to check for partial dependency. CD is a proper subset of a candidate key and it determines E, which is non-prime attribute.
So, given relation is also not in 2 NF.
So, the highest normal form is 1 NF.

- **Example 2 –** Find the highest normal form of a relation R(A,B,C,D,E) with FD set as {BC->D, AC->BE, B->E}

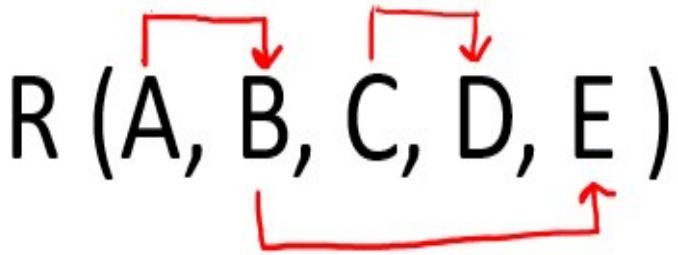
- Step 1. As we can see, (AC)+ ={A,C,B,E,D} but none of its subset can determine all attribute of relation, So AC will be candidate key. A or C can't be derived from any other attribute of the relation, so there will be only 1 candidate key {AC}.
  Step 2. Prime attributes are those attributes that are part of candidate key {A, C} in this example and others will be non-prime {B, D, E} in this example.
  Step 3. The relation R is in 1st normal form as a relational DBMS does not allow multi-valued or composite attribute.
  The relation is in 2nd normal form because BC->D is in 2nd normal form (BC is not a proper subset of candidate key AC) and AC->BE is in 2nd normal form (AC is candidate key) and B->E is in 2nd normal form (B is not a proper subset of candidate key AC).
  The relation is not in 3rd normal form because in BC->D (neither BC is a super key nor D is a prime attribute) and in B->E (neither B is a super key nor E is a prime attribute) but to satisfy 3rd normal for, either LHS of an FD should be super key or RHS should be prime attribute.
- So the highest normal form of relation will be 2nd Normal form.

- In this example, we have to find the highest normalization form, and for that, we are given a relation **R(A, B, C)** with functional dependencies as follows: {AB ->C, C ->B, AB ->B} Candidate Key (given): **{AB}**
- Clearly, prime attributes for Relation R are: {A,B} while non-prime attributes are: {C}.
- For this particular example, let us start from the order of hierarchy with higher restrictions, and firstly, we will check for BCNF here.
- Clearly, {AB->C} and {AB->B} are in BCNF because AB is the candidate key present on the LHS of both dependencies. The second dependency, {C->B}, however, is not in BCNF because C is neither a super key nor a candidate key.
- C->B is, however, present in 3NF because B is a prime attribute that satisfies the conditions of 3NF. Hence, relation R has 3NF as the highest normalization form.

- **Example 3:**

Given a relation R(A, B, C, D, E) and Functional Dependency set FD = { A → B, B → E, C → D}, determine whether the given R is in 2NF? If not convert it into 2 NF.

Solution: Let us construct an arrow diagram on R using FD to calculate the candidate key.



From above arrow diagram on R, we can see that an attributes AC is not determined by any of the given FD, hence AC will be the integral part of the Candidate key, i.e. no matter

- **Definition of 2NF:** No non-prime attribute should be partially dependent on Candidate Key
- Since R has 5 attributes: - A, B, C, D, E and Candidate Key is AC, Therefore, prime attribute (part of candidate key) are A and C while the non-prime attribute are B D and E

a. FD: **A → B** does not satisfy the definition of 2NF, as a non-prime attribute(B) is partially dependent on candidate key AC (i.e., key should not be broken at any cost).

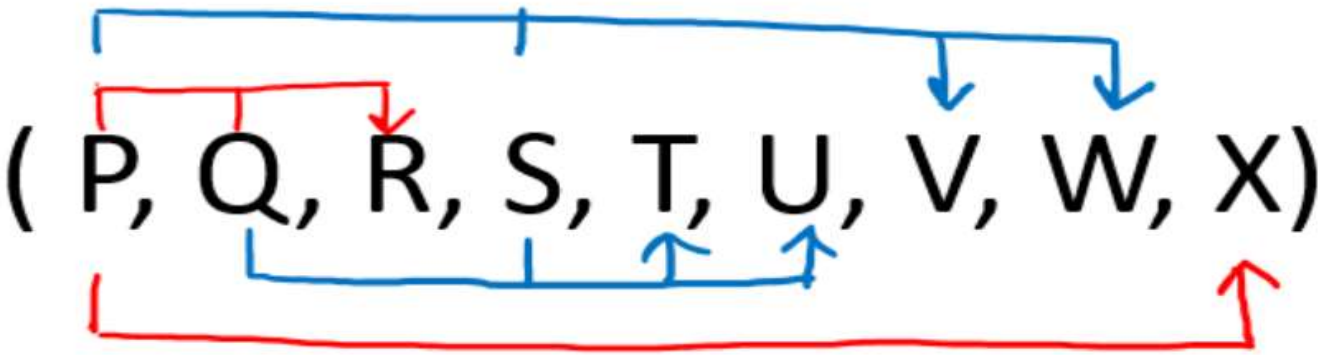
b. FD: **B → E** does **not violate** the definition of 2NF, as a non-prime attribute(E) is dependent on the non-prime attribute(B), which is not related to the definition of 2NF.

c. FD: **C → D** does not satisfy the definition of 2NF, as a non-prime attribute(D) is partially dependent on candidate key AC (i.e., key should not be broken at any cost)

- **Hence because of FD A → B and C → D, the above table R(A, B, C, D, E) is not in 2NF**
- **Convert the table R(A, B, C, D, E) in 2NF:**
- Since due to FD: A →B and C → D our table was not in 2NF, let's decompose the table
- **R1(A, B, E)** (from FD: A → B and B → E and both are violating 2 NF definition)
- **R2(C, D)** (Now in table R2 FD: C → D is Full F D, hence R2 is in 2NF)
- And create one table for candidate key AC
- **R3 (A, C)**

- **Example 4:** Given a relation R(P, Q, R, S, T, U, V, W, X) and Functional Dependency set FD = { PQ → R, QS → TU, PS → VW, and P → X }, determine whether the given R is in which normal form?

Solution: Let us construct an arrow diagram on R using FD to calculate the candidate key.



From the above arrow diagram on R, we can see that an attribute PQS is not determined by any of the given FD, hence PQS will be the integral part of the Candidate key, i.e. no matter what will be the candidate key, and how many will be the candidate key, but all will have PQS compulsory attribute.

Let us calculate the closure of PQS

PQS + = P Q R S T U X V W (from the closure method we studied earlier)

Since the closure of PQS contains all the attributes of R, hence **PQS is Candidate Key**

From the definition of Candidate Key (**Candidate Key is a Super Key whose no proper subset is a Super key**)

Since all key will have PQS as an integral part, and we have proved that PQS is Candidate Key, Therefore, any superset of PQS will be Super Key but not a Candidate key.

Hence there will be only **one candidate key PQS**

Since R has 9 attributes: - P, Q, R, S, T, U, V, W, X, and Candidate Key is PQS, Therefore, prime attributes (part of candidate key) are P Q and S while a non-prime attribute is R T U V W X

Given FD are { PQ → R, QS → TU, PS → VW, and P → X } and Super Key / Candidate Key is PQS

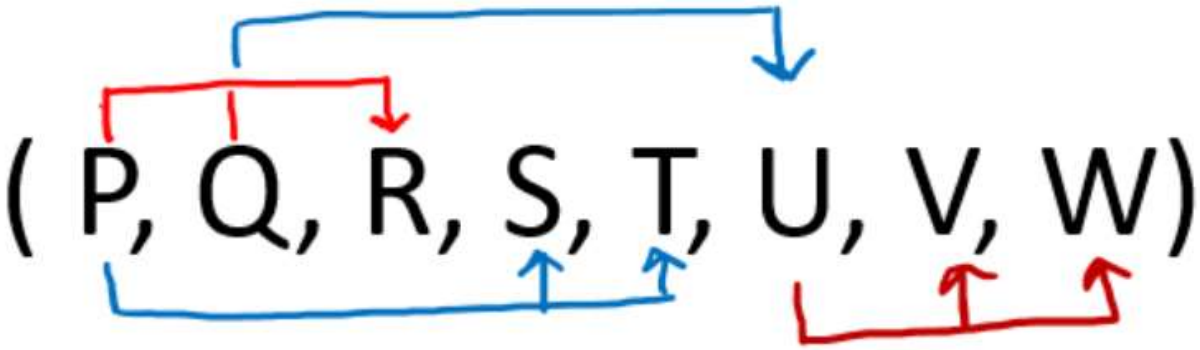
NOTE: To solve such questions, we apply reverse engineering, i.e. 1st check BCNF, if not then 3NF, if not then 2NF, and so on.

1. FD: **PQ** → **R** does not satisfy the definition of BCNF, as PQ is not Super Key, hence the table is not in BCNF (because if one dependency fails, all fails) now we check the same FD for 3NF.
2. FD: **PQ** → **R** even does not satisfy the definition of 3NF, as PQ is not Super Key or R is not a prime attribute, hence table is not in 3NF also (because if one dependency fails, all fails) now we check same FD for 2NF
3. FD: **PQ** → **R** even does not satisfy the definition of 2NF, as PQ is not Super Key and R which is not prime attribute depending on part of the key (partial dependency), hence table is not in 2NF also (because if one dependency fails, all fails).

Hence from the above three statements, we can say that table R (P, Q, R, S, T, U, V, W, X) is in 1NF only.

- **Example 5:** Given a relation R(P, Q, R, S, T, U, V, W) and Functional Dependency set FD = { PQ → R, P → ST, Q → U, and U → VW }, determine given R is in which normal form?

Solution: Let us construct an arrow diagram on R using FD to calculate the candidate key.



From the above arrow diagram on R, we can see that an attribute PQ is not determined by any of the given FD, hence PQ will be the integral part of the Candidate key, i.e. no matter what will be the candidate key, and how many will be the candidate key, but all will have PQ compulsory attribute.

Let us calculate the closure of PQ

PQ + = P Q R S T U V W (from the closure method we studied earlier)

Since the closure of PQ contains all the attributes of R, hence **PQ is Candidate Key**

From the definition of Candidate Key (**Candidate Key is a Super Key whose no proper subset is a Super key**)

Since all key will have PQ as an integral part, and we have proved that PQ is Candidate Key, Therefore, any superset of PQ will be Super Key but not Candidate key.

Hence there will be only **one candidate key PQ**

Since R has 8 attributes: - P, Q, R, S, T, U, V, W and Candidate Key is PQ. Therefore, prime attribute (part of candidate key) are P and Q while a non-prime attribute is R S T U V W

- Given FD are { PQ → R, P → ST, Q → U, and U → VW } and Super Key / Candidate Key is PQ

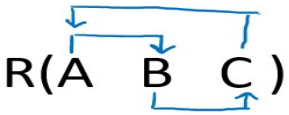
> **NOTE: To solve such questions, we apply reverse engineering, i.e. 1st check BCNF, if not then 3NF, if not then 2NF, and so on.**

1. FD: **PQ → R** satisfies the definition of BCNF, as PQ is Super Key, hence no need to check it for further normal forms, as it satisfies the highest one. Now we check another dependency in a reverse engineering manner.
2. FD: **P → ST** does not satisfy the definition of BCNF, as P is not Super Key, hence table is not in BCNF (because if one dependency fails, all fails) now we check the same FD for 3NF.
3. FD: **P → ST** even does not satisfy the definition of 3NF, as P is not Super Key or S T is not a prime attribute, hence table is not in 3NF also (because if one dependency fails, all fails) now we check same FD for 2NF.
4. FD: **P → ST** even does not satisfy the definition of 2NF, as P is not Super Key and S T which is not prime attribute depending on part of the key (partial dependency), hence table is not in 2NF also (because if one dependency fails, all fails).

Hence from the above three statements b, c, and d we can say that table R (P, Q, R, S, T, U, V, W,) is in 1NF only.

- Home work:

Given a relation R(A, B, C) and Functional Dependency set FD = { A → B, B → C, and C → A}, determine given R is in which normal form?



From the above arrow diagram on R, we can see that all the attributes are determined by all the attributes of the given FD, hence we will check all the attributes (i.e., A, B, and C) for candidate keys

Let us calculate the closure of A

A + = ABC (from the closure method we studied earlier)

Since closure A contains all the attributes of R, hence A is the Candidate key.

Let us calculate the closure of B

B + = BAC (from the closure method we studied earlier)

Since closure B contains all the attributes of R, hence B is the Candidate key.

Let us calculate the closure of C

C + = CAB (from the closure method we studied earlier)

Since closure C contains all the attributes of R, hence C is the Candidate key.

Hence three Candidate keys are: **A B and C**

- Since R has 3 attributes: - A B and C, Candidate Keys are A B and C, Therefore, prime attributes (part of candidate key) are A B C while there is no non-prime attribute
- Given FD are { A → B, B → C, and C → A } and Super Key / Candidate Key is A, B and C
- NOTE: To solve such questions, we apply reverse engineering, i.e. 1st check BCNF, if not then 3NF, if not then 2NF, and so on.

1. FD: **A → B** satisfy the definition of BCNF, as A is Super Key, we check other FD for BCNF
2. FD: **B → C** satisfy the definition of BCNF, as B is Super Key, we check other FD for BCNF
3. FD: **C -> A** satisfy the definition of BCNF, as C is Super Key

- **Since there were only three FD's and all FD: { A → B, B → C and C → A } satisfy BCNF, hence the highest normal form is BCNF.**
- **Therefore R(A, B, C) is in BCNF.**

- Thus , we can conclude that the following steps are followed to identify the normal form for a given relational schema.
- **STEP 1:** Calculate the Candidate Key of given R by using an arrow diagram and then using the closure of an attribute on R, such that from the calculated candidate key we can separate the prime attributes and non-prime attributes.
- **STEP 2:** Verify each FD's in the reverse engineering process, i.e. 1st check BCNF, if not then 3NF, if not then 2NF, and so on.
- **STEP 3:** If a table is in BCNF then it is in 3NF, 2NF, and 1NF, similarly if the table is in 3NF THEN it is in 2NF and 1NF. Hence, we can say that if a table is in the higher normal form then by default it is in lower normal form.
- **STEP 4:** Verify first FD with Definition of BCNF (First it should be in 3NF and if there exists a non-trivial dependency between two sets of attributes X and Y such that X → Y (i.e., Y is not a subset of X) then X is Super Key.
- **STEP 5:** If for any FD **STEP 5** fails(it signifies that table is not in BCNF), then verify that FD and remaining FD with Definition of 3NF(First it should be in 2NF and if there exist a non-trivial dependency between two sets of attributes X and Y such that X → Y (i.e. Y is not a subset of X) then X is Super Key or Y is a prime attribute
- **STEP 6:** If for any FD STEP 6 fails (it signifies that table is not in BCNF), then verify that FD and remaining FD with Definition of 2NF (No non-prime attribute should be partially dependent on the key of table).
- **STEP 7:** If for any FD STEP 7 fails (it signifies that table is not in 2NF), hence no need to check it for 1NF, as by default it is in 1NF.
- **STEP 8:** If all the FD's satisfy the definition of BCNF then we can say that given R is in BCNF, if any FD fails for BCNF and that FD and remaining FD satisfy for 3NF then we say R is in 3NF, similarly if any FD fails for 3NF and that FD and remaining FD satisfy for 2NF then we say R is in 2NF, otherwise the table is in 1NF.

- Next Topic
- UNIT 3